

Fine-Tuning and Adaptation: LoRA, Quantization and Forgetting

Unit 6 of the Generative AI and LLM crash course

GenAI crash course

2026-10-09

Contents

Session map	2
1. Terms: the term tree	3
2. Remember	4
3. Understand: the derivation	5
3.1 Tier 1 (easy): three levers, and the price of changing weights	5
3.2 Tier 2 (medium): data for SFT, and the LoRA idea	7
3.2.1 SFT data: format, masking, hygiene	7
3.2.2 Why not simply fine-tune every weight?	8
3.3 Tier 3 (medium-hard): why low rank, and why 4 bits	9
3.3.1 Why a low-rank update can be enough	9
3.3.2 Quantization: the 6 dB-per-bit rule	9
3.4 Tier 4 (hard): evaluating, overfitting, and when not to fine-tune	11
3.4.1 Small data overfits	11
3.4.2 Evaluating a fine-tuned model	11
3.4.3 When not to fine-tune	12
3.5 Tier 5 (hardest): open questions	12
4. Apply	13
4.1 Worked problems	13
4.2 Code and figures	13
5. Analyze	14
5.1 Limits	14

5.2 Which term dominates	14
5.3 Sensitivity	14
5.4 Failure modes	14
5.5 Uncertainty	14
6. Evaluate	15
7. Create	15
8. Visualization lesson	15
9. Problem set	16
10. Retrieval practice and recap	16
11. Where next	17

Session map

Outcome of this session

After this session you can:

1. choose between **prompting, retrieval and fine-tuning** for a task, and say what each can and cannot change;
2. prepare **supervised fine-tuning (SFT)** data: format, loss masking, validation, de-duplication and a leak-free split;
3. derive **LoRA**, count its parameters, and explain why a low-rank update can be enough (Eckart-Young);
4. explain **quantization** (uniform, block-wise, quantile) with the 6 dB-per-bit rule, and how QLoRA combines it with LoRA;
5. demonstrate and measure **catastrophic forgetting** and **overfitting on small data**, and say which mitigations worked and which did not.

- **Prerequisites:** Unit 2 (the Transformer block and its weight matrices, $12d^2$ parameters per block), Unit 3 (next-token loss), Unit 4 (the paired comparison and honest evaluation), Unit 5 (SVD appeared in LSA). A matrix-and-gradient comfort level; the key definitions are in Section 2.
- **Difficulty ladder used:** easy $\rightarrow$ medium $\rightarrow$ medium-hard $\rightarrow$ hard (Sections 3.1 to 3.4); the hardest tier is posed as open questions in Section 3.5.
- **Deliverables:** this document (.tex, .pdf, .docx), 6 figures, the module `codes/adapt/adapt_np/` (a small neural network with LoRA and hand-written backpropagation, low-rank tools, quantizers, SFT loss masking, data hygiene, a forgetting testbed; 10 tests including gradient checks), the *LoRA calculator* and *Quantization explorer* widgets, and a Colab notebook for the practical “fine-tune a small open model with LoRA”.

- **Source anchors:** Hu et al. 2021, arXiv:2106.09685 (LoRA: $W_0 + \Delta W = W_0 + BA$ with $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$; “random Gaussian initialization for A and zero for B ”; the parameter count $2 \times \hat{L}_{\text{LoRA}} \times d_{\text{model}} \times r$; with $r = 4$ on W_q and W_v of GPT-3 175B the checkpoint shrinks from 350GB to 35MB, and training memory from 1.2TB to 350GB; adapters can be merged with the frozen weights “introducing no inference latency”); Dettmers et al. 2023, arXiv:2305.14314 (QLoRA: 4-bit NormalFloat, double quantization saving about 0.37 bits per parameter, paged optimizers, a 65B model on one 48GB GPU); Biderman et al. 2024, arXiv:2405.09673 (“LoRA Learns Less and Forgets Less”); Kirkpatrick et al. 2017, arXiv:1612.00796 (catastrophic forgetting in neural networks). All IDs were checked and the quoted statements read in the PDFs on the day of writing.
- **What is measured here, and what is not.** The forgetting, rank, quantization and overfitting experiments run on **small NumPy networks and random weights** on this machine. They show mechanisms and orders of magnitude; they are **not** measurements of LLM fine-tuning. Claims about real LLMs are cited to the papers above. There is no GPU, no model and no `peft` library here, so the practical with a real model is a Colab notebook that was **not executed** while writing this document.

1. Terms: the term tree

Term	Symbol	One-line meaning	Where it appears	New / known
fine-tuning		continue training a pre-trained model on new data	3.1	new
supervised fine-tuning (SFT)		train on (instruction, response) pairs	3.2	new
instruction tuning		SFT on many tasks so the model follows instructions	3.2	new
loss masking		compute the loss on response tokens only	3.2	new
parameter-efficient fine-tuning (PEFT)		train a small number of added parameters, freeze the rest	3.2	new
LoRA, adapter	A, B, r	trainable low-rank update $W = W_0 + \frac{\alpha}{r}BA$	3.2	new
matrix rank	r	number of independent directions in a matrix	2	new
quantization		store weights with fewer bits	3.3	new
QLoRA, NormalFloat		LoRA on a frozen 4-bit model; quantile-based 4-bit code	3.3	new
catastrophic forgetting		learning a new task damages an old one	3.1	new
replay		mix some old data into the new training	3.1	new
model merging		combine adapters or models into one set of weights	3.3	new
overfitting / memorisation		fitting noise or copying the training data	3.4	known

Term	Symbol	One-line meaning	Where it appears	New / known
paired comparison		compare two systems on the same items	3.4	known

Terms *used but not taught here*: preference tuning and reinforcement learning from human feedback (named only), prefix and prompt tuning, mixture-of-experts.

2. Remember

Fine-tuning

Fine-tuning continues the training of a pre-trained model on a smaller, task-specific data set. It changes the **weights**. Prompting (Unit 4) and retrieval (Unit 5) change only the *input*.

SFT and loss masking

Supervised fine-tuning trains on pairs (instruction, response), usually stored as JSON lines such as {"instruction": "...", "input": "...", "response": "..."} . The text is joined with a chat template, and the loss of Unit 3 is computed **only on the response tokens** R :

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|R|} \sum_{t \in R} \ln p_{\theta}(x_t | x_{<t}). \quad (1)$$

Prompt tokens are context, not targets. **Instruction tuning** is SFT on a broad mix of tasks so the model learns to follow instructions in general.

Matrix rank

A matrix has **rank** r if it can be written as a sum of r outer products, $M = \sum_{i=1}^r u_i v_i^{\top}$, and no fewer. A rank-1 matrix has all rows proportional to one another. The **singular value decomposition** $M = U \Sigma V^{\top}$ lists the “strengths” $\sigma_1 \geq \sigma_2 \geq \dots$ of these directions; rank is the number of non-zero σ_i .

LoRA

For a frozen pre-trained weight $W_0 \in \mathbb{R}^{d \times k}$, **LoRA** (low-rank adaptation) learns an update $\Delta W = \frac{\alpha}{r} B A$ with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ and a small rank $r \ll \min(d, k)$:

$$h = W_0 x + \frac{\alpha}{r} B (A x). \quad (2)$$

A starts as small random Gaussian numbers and B as zeros, so $\Delta W = 0$ at the start (Hu et al. 2021). Only A and B are trained: $r(d + k)$ numbers instead of dk .

Quantization

Quantization stores each weight with b bits instead of 16 or 32. Uniform symmetric (absmax) quantization with $q_{\max} = 2^{b-1} - 1$ uses the scale $s = \max |w|/q_{\max}$ and stores $\text{round}(w/s)$ as an integer. A **block** of weights shares one scale.

Memory hooks.

- *Fine-tuning rewrites the brain; prompting and retrieval hand it a note.*
- *LoRA is a sticky note on each weight matrix:* the book (the model) stays as printed; the note is thin (rank r) and can be peeled off.
- *Quantization is rounding to the nearest tick on a ruler:* fewer ticks, smaller file, more rounding error, and one very long object stretches the ruler for everyone.
- *Forgetting: the new lesson overwrites the old chalk on the same blackboard.*

Recall quiz (answers in Section 10).

1. When is retrieval a better choice than fine-tuning, and when is it the other way round?
2. Why is the loss masked to response tokens?
3. How many trainable parameters does LoRA add to one 4096×4096 matrix at rank 8?
4. Roughly how much quality (in dB) does one extra bit buy in uniform quantization?
5. What was the one mitigation of forgetting that worked in every run here?

3. Understand: the derivation

3.1 Tier 1 (easy): three levers, and the price of changing weights

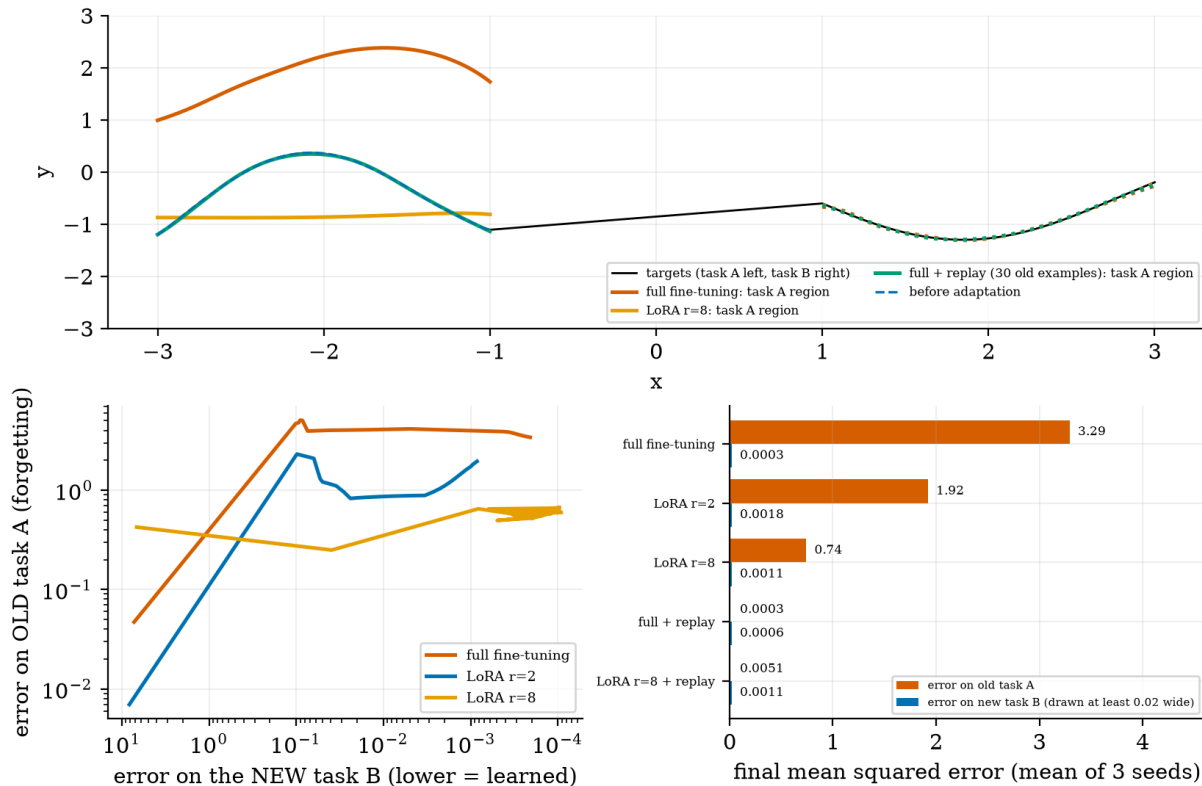
You have now met three ways to make a language model do what you want.

a rule of thumb, not a law: try prompting first, add retrieval for facts, fine-tune for behaviour. I did not measure this comparison here.

lever	what it changes	good for	weakness
prompting (Unit 4)	the input text	format, role, a few examples	limited by the context window
retrieval (Unit 5)	the evidence in the input	facts that are private, large or change often	needs an index; retrieval can fail
fine-tuning (this unit)	the weights	style, format, a skill, a domain's vocabulary, a consistent behaviour	needs data and compute; can damage other abilities

The price of changing weights is the subject of the unit's central picture. I trained a small network ($1 \rightarrow 64 \rightarrow 64 \rightarrow 1$, tanh) on **task A**, a smooth function on $x \in [-3, -1]$, until its error was 4.6×10^{-5} . Then I adapted it to **task B**, a different function on $x \in [1, 3]$, using only 30 examples, and measured the error on *both* tasks.

Catastrophic forgetting is what the red curve shows: full fine-tuning learns task B almost perfectly (error 3×10^{-4}) but the error on task A rises from 4.6×10^{-5} to **3.29** (mean of 3 seeds; range 1.49 to



Top: the targets (black: task A on the left, task B on the right), the network before adaptation (blue dashed, which sits on task A), and after adaptation to task B by full fine-tuning (red: it fits B, shown dotted on the right, but its prediction on task A has moved far away), by LoRA with rank 8 (orange: A is less damaged, but still flattened) and by full fine-tuning with 30 old examples mixed in (green: both tasks fit). Bottom left: forgetting (error on A) against learning (error on B, axis reversed) during adaptation, median of 3 seeds. Bottom right: final errors, mean of 3 seeds.

5.00): the network no longer knows the old task at all. LoRA with rank 2 ended at 1.92 (range 0.89 to 2.80), rank 8 at 0.74 (range 0.53 to 1.08). **Replay**, mixing 30 old examples into the 30 new ones, kept task A almost intact (error 0.0003 for full fine-tuning, 0.0051 for LoRA rank 8) while learning B just as well (0.0006 and 0.0011).

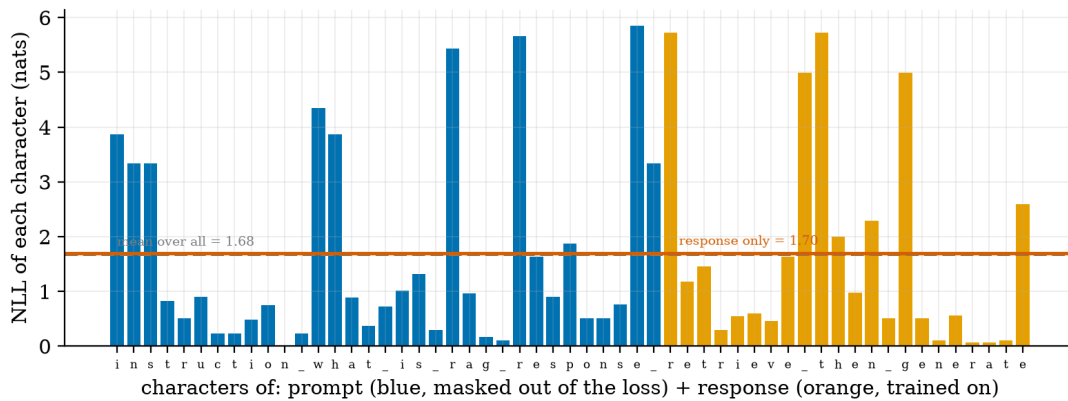
Read the comparison honestly:

- At *equal learning of B* (the first time the error on B drops below 0.05), the error on A was 4.66, 2.81 and 3.94 for full fine-tuning (three seeds), 1.21, 2.13 and 0.55 for LoRA rank 2, and 0.25, 0.24 and 0.30 for rank 8. In this toy LoRA forgot less, and a higher rank (8) forgot less than rank 2.
- But it is **fragile**: LoRA rank 2 at three learning rates (0.003, 0.01, 0.03) ended with mean A-errors of 1.92, 3.78 and 0.83, and single seeds ranged from 0.26 to 7.49. With 3 seeds I would not claim that rank 2 beats full fine-tuning.
- The result agrees in *direction* with Biderman et al. (2024), whose title is their finding on large models (“LoRA learns less and forgets less”; with rank 16 LoRA “learns less and forgets less than all other models”, and rank 256 “learns as much as the other methods while forgetting less”). Their setting is language-model training at a very different scale; mine is a 4353-parameter toy.
- The only mitigation that worked in every run was **replay**.

3.2 Tier 2 (medium): data for SFT, and the LoRA idea

3.2.1 SFT data: format, masking, hygiene

A typical record is {"instruction": "Summarise in one line.", "input": "<text>", "response": "<summary>"}. A chat template turns it into one string; the model is trained to predict the response part given the rest, with the loss (1).



Per-character negative log-likelihood of the string “instruction what is rag response retrieve then generate” under a character 4-gram model trained on the course syllabus (Unit 1). Blue bars are the 33 prompt characters, which are masked out of the SFT loss; orange bars are the 22 response characters, which are trained on. Mean over all characters 1.68 nats; response only 1.70; prompt only 1.67.

The three averages in Figure 2 are almost equal, so **masking changes the loss value little here**; its effect is on *what is trained*. Without masking, 33 of the 55 characters, 60% of the training signal, would be spent on learning to predict the user’s prompt, i.e. to imitate the questions instead of answering them. With masking, all of it goes to the response. (Whether to mask is a design choice: masking

prompt tokens is common practice, but it is not universal and I did not measure its effect on a real model.)

Data hygiene matters as much as the algorithm, because a few hundred examples set the behaviour. The module provides three checks:

- `validate_sft_record` lists *every* problem of a record (empty instruction, empty response, a response of fewer than two words);
- `near_duplicates(texts, threshold)` finds pairs whose word sets have Jaccard similarity $\geq$ threshold. Example: “the cat sat on the mat” and “the cat sat on a mat” share 5 of 6 distinct words: $5/6 = 0.833$;
- `split_by_hash(records, key, val_frac)` assigns each record to training or validation by a hash of its prompt, so the same prompt (or a copy of it) can never be on both sides. A random split would let near-copies leak into the validation set and flatter the model.

3.2.2 Why not simply fine-tune every weight?

Full fine-tuning stores a gradient and two Adam statistics for **every** parameter. Hu et al. report that for GPT-3 175B, LoRA reduced the training memory from 1.2TB to 350GB and, with $r = 4$ on W_q and W_v , shrank the saved checkpoint from 350GB to 35MB (about 10,000 times). The saved file matters in practice: one frozen base model can serve many tasks, each with its own tiny adapter.

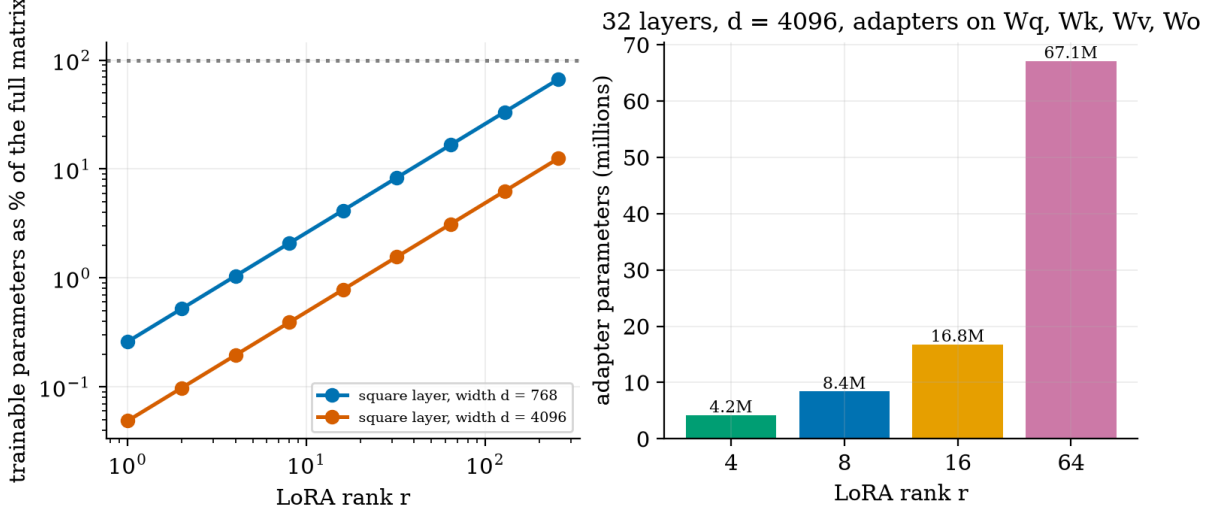
The LoRA gradient (derivation). Let $W = W_0 + sBA$ with $s = \alpha/r$, and let $G = \partial\mathcal{L}/\partial W$ be the gradient with respect to the *effective* weight (the same G that full fine-tuning would compute). A small change dB changes W by $s dB A$, so $d\mathcal{L} = \text{tr}(G^\top s dB A) = \text{tr}((sGA^\top)^\top dB)$. Hence

$$\frac{\partial\mathcal{L}}{\partial B} = s G A^\top, \quad \frac{\partial\mathcal{L}}{\partial A} = s B^\top G. \quad (3)$$

(the second by the same argument with dA). Two consequences. (1) At the start $B = 0$, so $\partial\mathcal{L}/\partial A = 0$: the first updates move only B ; this is why B is initialised to zero and A randomly, not both zero (then neither could ever move). (2) The adapted network starts *exactly* equal to the base network. The module’s tests check both: `test_backprop_matches_finite_differences_full_and_lora` compares (3) with numerical differentiation, and `test_lora_starts_identical_and_adapter_off_restores_base` checks that with $B = 0$ the output is unchanged, and that after training, switching the adapter off returns the base network’s output *exactly*.

Counting parameters. One adapter on a $d_{\text{out}} \times d_{\text{in}}$ matrix has $r(d_{\text{out}} + d_{\text{in}})$ numbers, against $d_{\text{out}}d_{\text{in}}$: a share of $r(d_{\text{out}} + d_{\text{in}})/(d_{\text{out}}d_{\text{in}}) = 2r/d$ for a square matrix. For $d = 4096$ and $r = 8$: 65,536 numbers against 16,777,216, i.e. 0.39%. For a stack of 32 layers with $d = 4096$ the four attention matrices hold $32 \cdot 4 \cdot 4096^2 = 2.15 \times 10^9$ weights; adapters of rank 8 on all four have 8.39×10^6 numbers (0.39% of them), and on W_q and W_v only, as in Hu et al., 4.19×10^6 (the formula $|\Theta| = 2\hat{L}_{\text{LoRA}} dr$ with $\hat{L}_{\text{LoRA}} = 64$ adapted matrices).

Merging for serving. Because $W = W_0 + sBA$ is linear, the adapter can be folded into the weights once training is done, $W_{\text{merged}} = W_0 + sBA$, so inference runs at the speed of the base model (Hu et al.: “no inference latency”). Two adapters can be summed: $W_0 + s_1 B_1 A_1 + s_2 B_2 A_2 = W_0 + [s_1 B_1 \ s_2 B_2] \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$, a single update of rank at most $r_1 + r_2$. Whether the *summed* behaviour is useful depends on the tasks; averaging models fine-tuned from one base is an active technique that I mention only as an overview.



Left: LoRA’s trainable share of a square matrix against the rank, for widths 768 and 4096 (the share is exactly $2r/d$, linear in r). Right: adapter size for a hypothetical 32-layer stack with $d = 4096$ and adapters on all four attention matrices: 4.2M, 8.4M, 16.8M and 67.1M numbers for ranks 4, 8, 16 and 64.

3.3 Tier 3 (medium-hard): why low rank, and why 4 bits

3.3.1 Why a low-rank update can be enough

Hu et al. hypothesise that “the updates to the weights also have a low intrinsic rank during adaptation”. The mathematical fact behind the idea is the **Eckart-Young theorem** (quoted, not re-derived): among all matrices of rank at most r , the one closest to M in Frobenius norm is the truncated SVD $\sum_{i \leq r} \sigma_i u_i v_i^\top$, and its error is

$$\min_{\text{rank } X \leq r} \|M - X\|_F = \sqrt{\sum_{i > r} \sigma_i^2}. \quad (4)$$

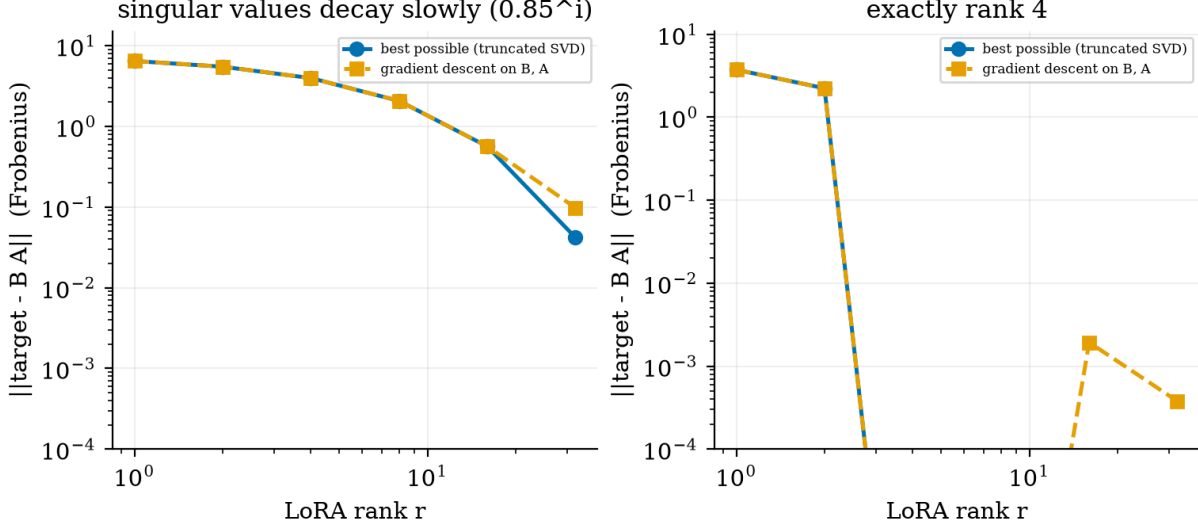
So how good a rank- r update can be depends on how fast the singular values of the *ideal* update fall. Figure 4 checks that plain gradient descent on B and A (the way LoRA trains) really finds this optimum.

Read it as follows. If the update the task needs is **exactly** low rank (right), a rank-4 adapter loses nothing. If its singular values decay slowly (left), every rank is a compromise: rank 8 leaves an error of 2.07 out of 7.59. Whether real fine-tuning updates are close to low rank is an *empirical* question about real models; Hu et al. report evidence for it, and I have shown only the matrix mathematics. A caution: here the target matrix is given; in fine-tuning the “ideal update” is defined by the data and the loss, and gradient descent finds an optimum of *that* problem.

3.3.2 Quantization: the 6 dB-per-bit rule

With the absmax rule of Section 2, rounding error is roughly uniform in $[-s/2, s/2]$, with variance $s^2/12$, where $s = a/q_{\max}$ and $a = \max |w|$. For weights of standard deviation σ , the signal-to-quantization-noise ratio is

$$\text{SQNR} = \frac{\sigma^2}{s^2/12} = \frac{12 q_{\max}^2 \sigma^2}{a^2} \Rightarrow 10 \log_{10} \text{SQNR} = 10.8 + 20 \log_{10} q_{\max} - 20 \log_{10} \frac{a}{\sigma} \text{ dB}. \quad (5)$$



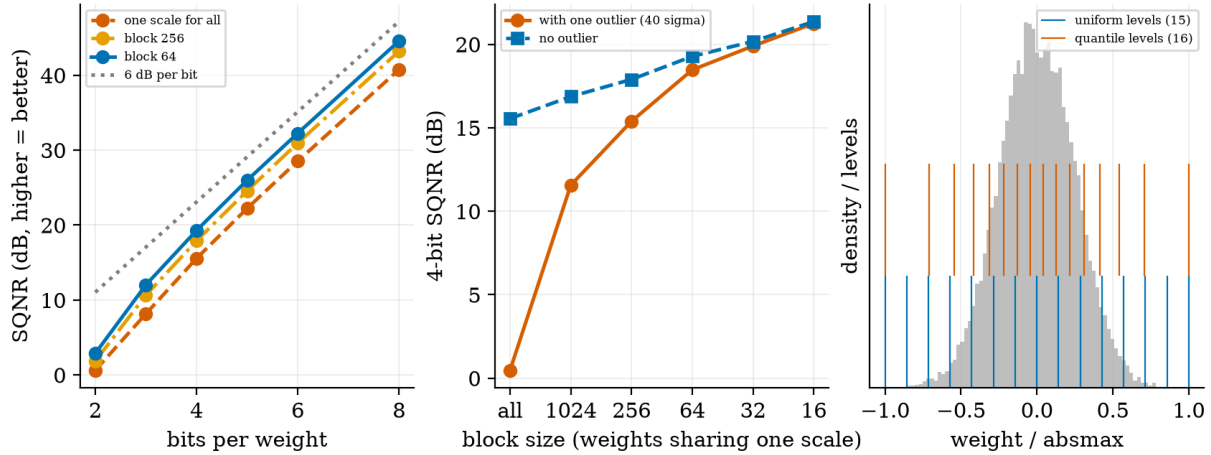
Error of the best rank- r approximation of a target matrix (blue, (4)) and of what gradient descent on B, A reaches (orange), for two 64×64 targets. Left: singular values decaying like 0.85^i (the matrix norm is 7.59): errors 6.45, 5.49, 3.96, 2.07, 0.564, 0.042 at ranks 1, 2, 4, 8, 16, 32; gradient descent matches up to rank 16 and reaches 0.098 at rank 32 (it had not converged in 5000 steps). Right: an exactly rank-4 target: errors 3.74 and 2.24 at ranks 1 and 2, and zero from rank 4 on (gradient descent: $\leq 2 \times 10^{-3}$).

Each extra bit doubles $q_{\max} \approx 2^{b-1}$, which adds $20 \log_{10} 2 = 6.02$ dB. **Worked check:** for the 16,384 Gaussian weights used here, $a/\sigma = 4.04$; for $b = 4$, $q_{\max} = 7$, so (5) gives $10.8 + 16.9 - 12.1 = 15.6$ dB; the module measures 15.55 dB (and for $b = 8$: predicted 40.74 dB, measured 40.75 dB). The measured values with one scale for all weights are 0.6, 8.2, 15.6, 22.2, 28.5, 40.7 dB for 2, 3, 4, 5, 6, 8 bits: about 6 to 7 dB per bit.

Blocks and outliers. With *blocks* of B weights each block’s absmax is the maximum of only B values, which is smaller than for the whole tensor: for $B = 64$ the average block maximum is 2.6σ (against 4.04σ for the whole tensor), so (5) predicts 19.4 dB, a gain of about 3.8 dB; the measured SQNR at 4 bits goes from 15.6 dB (one scale) to 17.9 (blocks of 256) and 19.3 (blocks of 64). The cost is one 16-bit scale per block: $b/8 + 2/B$ bytes per weight, i.e. 0.53 bytes for $b = 4$, $B = 64$, which is 3.76 times smaller than 16-bit.

One outlier is enough to ruin per-tensor quantization. Put a single weight of size 40σ in the tensor. With one scale, $s = 40\sigma/7 = 5.7\sigma$, larger than almost every weight, so nearly all weights round to 0 and the noise equals the signal: SQNR ≈ 0 dB. Measured: 0.5 dB ((5)’s uniform-noise assumption fails here). With blocks of 64 the outlier ruins only its own block: 18.5 dB; with blocks of 16, 21.3 dB, the same as without the outlier (21.4). Real LLMs do have large outlier weights and activations, which is why block-wise scales are used.

Quantile levels (NormalFloat-like). Uniform levels waste resolution in the tails, where there are few weights, and are coarse in the middle, where most are. A **quantile code** places the 2^b levels at the quantiles $(i + \frac{1}{2})/2^b$ of the standard normal and rescales to $[-1, 1]$: more levels where the weights are dense (Figure 5, right). On Gaussian weights with blocks of 64 at 4 bits the quantile code reaches 20.7 dB against 19.3 dB for uniform steps. This is a *simplified* version of QLoRA’s **NF4**, which Dettmers et al. describe as a data type that is “information theoretically optimal for normally distributed weights” and “yields better empirical results than 4-bit Integers and 4-bit Floats” (their code differs in detail, for example in how zero is represented).



Left: SQNR (dB, higher is better) against bits per weight for 16,384 Gaussian weights, with one scale for all weights, blocks of 256 and blocks of 64, and the line of 6 dB per bit. Middle: 4-bit SQNR against block size, with and without a single outlier of 40σ . Right: the 15 uniform levels and the 16 quantile levels over the weight histogram.

QLoRA keeps the base model frozen in 4-bit NF4, adds 16-bit LoRA adapters, and back-propagates through the dequantized weights into the adapters. It also uses **double quantization** (quantizing the per-block scales, saving about 0.37 bits per parameter) and paged optimizers; with them Dettmers et al. fine-tune a 65B-parameter model on a single 48GB GPU. Memory arithmetic for a 7-billion-parameter model: 16-bit weights take $7 \times 10^9 \times 2 = 14$ GB; 4-bit with one 16-bit scale per 64 weights takes $7 \times 10^9 \times 0.53125 = 3.7$ GB, before the adapters and activations.

3.4 Tier 4 (hard): evaluating, overfitting, and when not to fine-tune

3.4.1 Small data overfits

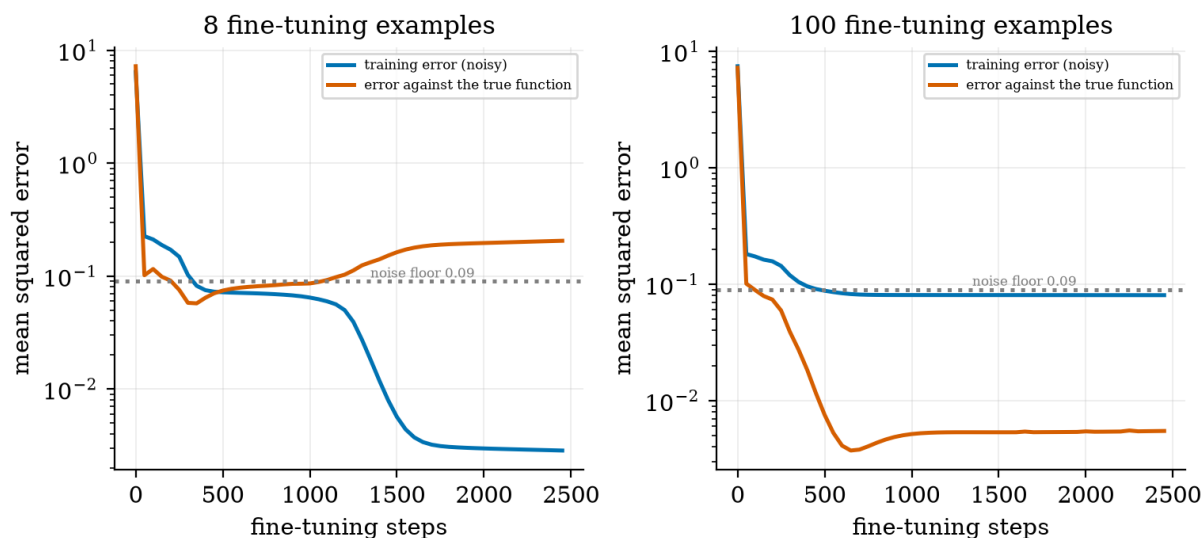
Fine-tuning on a few examples is the common case, and it is where it fails most easily. I fine-tuned the same network to task B with noisy targets (noise variance 0.09), with 8 and with 100 examples, and tracked the error against the *true* function (which a real project cannot see; here I can).

The lesson is the one from Unit 1 and Unit 4: **a training error far below the noise floor is a warning, not a success.** Keep a held-out validation set (split by hash, Section 3.2.1), stop when it stops improving (early stopping), prefer more data to more steps, and consider a lower rank.

3.4.2 Evaluating a fine-tuned model

Four checks, in this order:

1. **Held-out task performance** against the *un-tuned* model on the same prompts, with the paired comparison and its interval from Unit 4 (a gain on 100 items is rarely significant).
2. **Regression test on old abilities:** keep a small fixed set that measures what you must not lose (task A in Figure 1) and re-measure it after every adaptation.
3. **Overfitting check:** training against validation loss (Figure 6).
4. **Human or task-specific review** of outputs, because a lower loss does not guarantee better answers (Unit 9).



Fine-tuning error against steps with 8 examples (left) and 100 examples (right): training error against the noisy targets (blue) and error against the true function (red). The dotted line is the noise floor 0.09. With 8 examples the error against the true function is best at 0.057 (step 350) and ends at 0.205, 3.6 times worse, while the training error keeps falling to 0.0028, far below the noise floor: the network is fitting noise. With 100 examples the best true error is 0.0037 (step 650), the end value is 0.0055, and the training error sits at the noise floor (0.080).

3.4.3 When not to fine-tune

- The need is **facts that change**: use retrieval (Unit 5).
- A **good prompt** with a few examples already works (Unit 4): fine-tuning adds cost and maintenance.
- You have **fewer than a few hundred good examples** (Figure 6).
- You cannot afford a **regression test** for forgetting.
- You need to **explain or audit** where an answer came from: retrieval gives sources; fine-tuned weights do not.
- The base model will be replaced soon: adapters trained on it are lost.

3.5 Tier 5 (hardest): open questions

1. **Why does LoRA forget less?** One idea is that a low-rank update changes the function less. Figure 1 shows the direction but also that it is fragile (rank 2 at three learning rates gave 1.92, 3.78 and 0.83). Which factors control it in large models is an active research question (Biderman et al.).
2. **How low is the intrinsic rank of fine-tuning?** It depends on the task: style and format changes may be low-rank, while teaching a model a lot of *new* knowledge may not be. I did not test it.
3. **How should forgetting of a large model be measured?** A fixed regression set covers only the abilities you thought of. There is no complete answer.
4. **Fine-tuning versus retrieval for knowledge.** Which is more reliable for adding facts is still debated; measure it on your own task.

4. Apply

4.1 Worked problems

Problem 1 (LoRA parameters). $d = 4096$, one square matrix, rank 8: $8 \cdot (4096 + 4096) = 65,536$ numbers, 0.39% of 16,777,216.

Problem 2 (adapter on a layer stack). 32 layers, adapters of rank 8 on W_q and W_v : $32 \cdot 2 \cdot 65,536 = 4,194,304$ numbers. Against the $12d^2L = 6.44 \times 10^9$ weights of the stack, that is 0.065%.

Problem 3 (quantization memory). 7×10^9 weights: 16-bit: 14 GB; 4-bit with block 64 and 16-bit scales: $7 \times 10^9 \times (0.5 + 2/64) = 3.72$ GB, $3.76\times$ smaller.

Problem 4 (SQNR). 8-bit, $a/\sigma = 4.04$: $q_{\max} = 127$, $\text{SQNR} = 10.8 + 42.1 - 12.1 = 40.8$ dB; measured per-tensor 40.75 dB.

Problem 5 (is the fine-tune better?). Base model 62% on a 100-item test set, fine-tuned 69%; disagreements: $b = 15$ items only the tuned model gets right, $c = 8$ only the base. From Unit 4 the 95% interval is $[-0.023, 0.163]$: not distinguishable. The set needs about 450 to 500 items for a 5-point gain.

4.2 Code and figures

Everything is in `codes/adapt/adapt_np/` (see its README). A LoRA network and its training:

```
net = Net([1, 64, 64, 1], rng)           # pre-trained on task A (mode "full")
net.add_lora(r=8, alpha=16, rng=rng)      # B = 0, A ~ Gaussian: same function as
before
train(net, XB, yB, steps=1500, lr=1e-2, mode="lora") # only A and B move
mse(net, gA, fA(gA), adapter=False)        # adapter off: the base network, exactly
```

Replay is one argument: `train(..., X_extra=XA[:30], y_extra=yA[:30])`. Quantization: `quantize_uniform(w, bits=4, block=64)` and `quantize_codebook(w, normalfloat_levels(4), block=64)`; `sqnr_db(w, wq)` measures the result.

For the real practical, **fine-tune a small open-source model with LoRA**, open the Colab notebook `u6_lora_real_model.ipynb`. It builds a small SFT data set, masks prompt tokens, attaches LoRA adapters with the `peft` library, trains, merges, and runs the regression test of Section 3.4.2. It was **not executed** while writing this document; the base model it names (Qwen/Qwen2.5-0.5B-Instruct) was confirmed to exist on the Hugging Face hub, but the names of its attention modules in the notebook are assumed and should be checked by printing the model.

Try it in the playground

The **LoRA calculator** shows the adapter size and its share of the model for any width, depth, rank and choice of matrices. The **Quantization explorer** shows each weight before and after quantization, with the quality in dB and the memory saved; try one outlier with one scale, then blocks of 16.

5. Analyze

Checklist items used: limits (5.1), dominance (5.2), sensitivity (5.3), failure modes (5.4), uncertainty (5.5).

5.1 Limits

- $r \rightarrow \min(d, k)$: LoRA can represent any update; the share $2r/d \rightarrow 2$ for square matrices, i.e. *more* parameters than the full matrix (at $r = d/2$ the adapter equals the matrix size). Use $r \ll d$.
- $B = 0$, A arbitrary: $\Delta W = 0$.
- $b \rightarrow 16$ bits: SQNR about $6 \cdot 16 - 1 \approx 95$ dB: lossless for practical purposes.
- Block size 1 with one scale per weight: lossless, but the scales cost 16 bits per weight.

5.2 Which term dominates

Adapter memory is dominated by $r(d_{\text{in}} + d_{\text{out}})$: linear in rank. Training memory for a *full* fine-tune is dominated by the optimizer’s two extra numbers per parameter; for LoRA those apply only to the adapter (Hu et al.’s 1.2TB to 350GB).

5.3 Sensitivity

Forgetting is **very sensitive** to the learning rate and seed in the toy (rank 2 at three learning rates gave 1.92, 3.78, 0.83; a single seed gave 7.49). Quantization quality is most sensitive to the outliers and the block size (Figure 5, middle). Overfitting is most sensitive to the number of examples (Figure 6).

5.4 Failure modes

- **Forgetting** of general ability (Figure 1).
- **Overfitting** on a small set (Figure 6).
- **Training on the prompt** because the loss was not masked (Figure 2).
- **Leakage** between train and validation through near-duplicates (Section 3.2.1).
- **One outlier wrecking a quantized tensor** (Figure 5).
- **Adapter and base model mismatch**: an adapter only works with the exact base model it was trained on.

5.5 Uncertainty

Everything numerical here comes from a 4353-parameter toy, 3 seeds and one random tensor family. The orderings (rank 8 less forgetting than full fine-tuning; replay fixes forgetting) held in all 3 seeds; the ordering of rank 2 did not. The quantization numbers are for Gaussian weights; real weights are not exactly Gaussian.

6. Evaluate

Methods compared. *Full fine-tuning*: most flexible, most expensive, most forgetting in the toy. *LoRA*: tiny adapters, merge for serving, less forgetting in the toy (with caveats). *QLoRA*: LoRA on a 4-bit base: smallest memory, at the price of quantization error and a slower step. *Prompting and retrieval*: no training at all.

Cost versus accuracy. The adapter’s quality depends on rank (Figure 4), the base’s quality on bits (Figure 5); memory goes down with both. Choose the smallest rank and the most bits that pass *your* evaluation (Section 3.4.2).

When not to use. Section 3.4.3.

Wider context. Adapting a model is closer to *teaching* than to *programming*: you shift tendencies, you cannot guarantee behaviour. That is why the regression test, the validation set and the paired comparison are not optional extras.

7. Create

What you can build from this

1. **Forgetting lab:** repeat Figure 1 with 10 seeds and report the median and interval of the A-error for full fine-tuning, LoRA rank 2, 8 and 32, with replay fractions 0%, 5% and 20%.
2. **Rank-needed estimator:** given an update matrix ΔW , compute its singular values and the smallest rank that keeps 95% of $\|\Delta W\|_F^2$.
3. **Quantization benchmark:** find the smallest number of bits and block size that keep the SQNR above 20 dB on your own weight tensor, and the memory you save.
4. **SFT data checker:** a command-line tool that validates a JSONL file, flags near-duplicates, and writes a hash-based split.
5. **Combine with Unit 5:** fine-tune a small model to answer *only* from the retrieved passages and compare with prompting alone, using the paired interval of Unit 4.
6. **Module for the open-source package:** add LoRA merging of two adapters (the identity of Section 3.2.2) with a test that the merged update equals the sum.

8. Visualization lesson

charts taught here. How to read Figure 1’s bottom-left panel: the *higher* a curve stays when it moves to the *right* (learning B), the more it forgets; a curve that moves right while staying low forgets little.

Chart type	Best for	How it misleads	matplotlib call
trajectory plot (error A against error B)	a trade-off over training time (Fig. 1 bottom left)	median across seeds hides spread; reversed axis can confuse	<code>ax.loglog(errB, errA); ax.invert_xaxis()</code>
horizontal grouped bar with value labels	comparing methods on two metrics (Fig. 1 bottom right)	tiny bars vanish unless labelled	<code>ax.barh(y-.2, a, .4); ax.text(...)</code>

Chart type	Best for	How it misleads	matplotlib call
per-token bar chart	what a loss is made of (Fig. 2)	colour implies importance of one group	<code>ax.bar(i, nll, color=...)</code>
log-log line with reference	scaling in rank or bits (Fig. 3, Fig. 5)	straight line only means a power law if the data are	<code>ax.loglog(r, frac)</code>
exact versus achieved (two lines)	checking an algorithm against a theorem (Fig. 4)	convergence budget changes the gap (rank 32)	<code>ax.semilogy(r, opt); ax.semilogy(r, gd, "s--")</code>
train/validation curve	overfitting (Fig. 6)	needs a held-out set that is truly separate	<code>ax.plot(steps, train); ax.plot(steps, true)</code>

9. Problem set

Graded exercises

1. (easy) A LoRA adapter has rank 4 on a 768×768 matrix. How many numbers does it have, and what share of the matrix is that?
2. (medium) Show that if $B = 0$ and $A = 0$ are *both* initialised to zero, LoRA never learns, using (3).
3. (medium-hard) Using (5), predict the 4-bit SQNR for $a/\sigma = 3$ and compare with 15.6 dB for $a/\sigma = 4.04$. Then explain why block-wise quantization raises the SQNR.
4. (hard) A target matrix has singular values 5, 3, 1, 0.5, 0.1, 0.1. Compute the best rank-1, rank-2 and rank-3 approximation errors with (4), and the smallest rank for which the error is below 0.2.
5. (hardest) Two adapters (B_1, A_1) and (B_2, A_2) of rank r are summed into one weight update. Show the sum has rank at most $2r$ by writing it as one product, and say why *averaging* two models fine-tuned from the same base can still behave unpredictably even though this identity is exact.

10. Retrieval practice and recap

Answers to the Section 2 quiz.

1. Retrieval is better when the need is facts that are large, private or change often; fine-tuning is better for a consistent behaviour, format or skill.
2. The prompt is context, not the target; unmasked, 60% of the signal in Figure 2 would train the model to predict the user's text.
3. $8 \cdot (4096 + 4096) = 65,536$ numbers.
4. About 6 dB per extra bit (Equation (5)); measured 6 to 7 dB).
5. Replay, mixing a few old examples into the new data.

Explain it to a friend.

- Why does LoRA start with $B = 0$ and A random, not both zero?
- Why can one huge weight ruin 4-bit quantization of a whole tensor, and what fixes it?
- Why is a training error far below the noise floor bad news?

Re-derive tomorrow without looking: (a) (3); (b) the 6 dB-per-bit rule; (c) the rank- r approximation error (4) for a given list of singular values.

11. Where next

- **Next:** Unit 7 (multimodal and generative models: diffusion, image-text embeddings, speech), which uses the diffusion idea sketched in Unit 1 and the fine-tuning of this unit.
- **Terms met but not taught:** preference tuning (RLHF, DPO), prefix and prompt tuning, structured pruning, distillation.
- **Open threads:** why LoRA's forgetting is so seed- and learning-rate dependent in the toy (Section 3.5); whether the real notebook reproduces the direction of Figure 1.